A6: Build and Integrate an AI Decision Capability

Course: Platform Development
Week: 08 — AI-Native Capability Design
Student Name: Sawitta Thiabsaeng
Student ID: 6731503123
Project / Capability Name: Support Request Classifier
Implementation Method: Jev Playground and Python Fallback Demonstration

1. Problem and Capability

Problem

Support teams receive requests written in natural language. Identifying whether a request concerns an account problem, a technical problem, or a general inquiry can take time when the process is performed manually.

The Support Request Classifier uses an AI decision model to categorize support messages and help determine the appropriate next handling action.

Why AI?

Users may describe the same problem in different ways, use informal language, or include multiple issues in one message. An AI model can interpret natural-language descriptions that may not match a fixed set of keywords.

However, AI decisions can be incorrect or uncertain. Therefore, the system also needs deterministic validation and fallback rules to handle vague messages, invalid outputs, model failures, and ambiguous cases.

Input

The capability receives a text message from a user requesting support.

Example:

> I cannot log in because the password reset email never arrives.

Expected Output

The model selects one of three predefined categories:

- `ACCOUNT_ISSUE` — Account-related problems.
- `TECHNICAL_ISSUE` — Technical problems.
- `GENERAL_INQUIRY` — General questions or requests for information.

The application should validate the model's decision before routing the request. If the output is invalid or the message is ambiguous, the fallback layer can return a review status and an appropriate handling action.

2. Decision Implementation

The AI decision capability was implemented using Jev Playground with a Choice decision containing three possible categories.

The question used for the five test cases was:

What category best describes this support request?

Test Results

Test	Input Summary	Expected Handling	Actual Jev Decision	Result
T01	Cannot log in; password reset email does not arrive	`ACCOUNT_ISSUE`	`ACCOUNT_ISSUE`	Match
T02	Website shows a 500 Internal Server Error	`TECHNICAL_ISSUE`	`TECHNICAL_ISSUE`	Match
T03	Asks about support opening hours	`GENERAL_INQUIRY`	`GENERAL_INQUIRY`	Match

T04	Cannot access an account and reports a website error	SEND_TO_HUMAN	ACCOUNT_ISSUE	Fallback required
T05	“It does not work. Please help.”	ASK_FOR_MORE_INFORMATION	TECHNICAL_ISSUE	Fallback required

The results show that Jev classified the three clear requests as expected. However, T04 and T05 demonstrate that a model can produce a category even when a request contains multiple issues or insufficient information.

For T04, the message contains both account-related and technical problem indicators. The fallback should send the request for human review rather than automatically routing it.

For T05, the message does not provide enough information to identify the actual problem. The fallback should ask the user for more details.

These cases demonstrate why the model's output should be validated before the application takes action.

Actual Model Execution Evidence

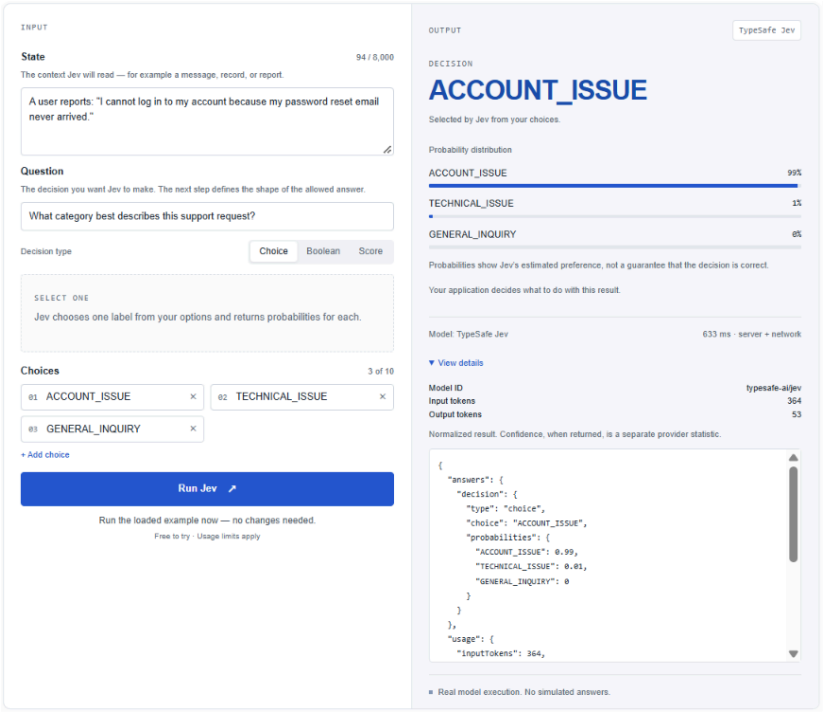


Figure 1. Jev classifies an account access and password recovery request as ACCOUNT_ISSUE.

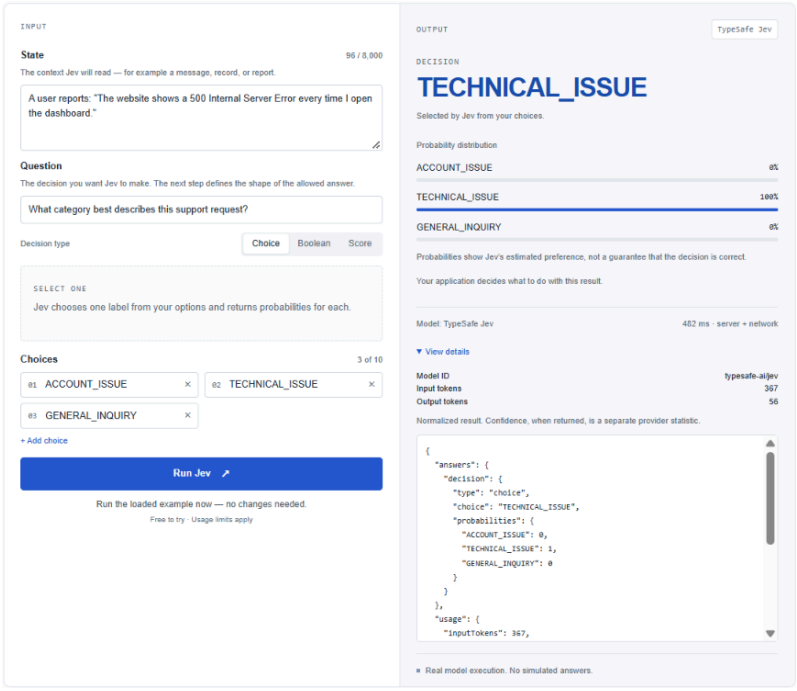


Figure 2. Jev classifies a website 500 error as TECHNICAL_ISSUE.

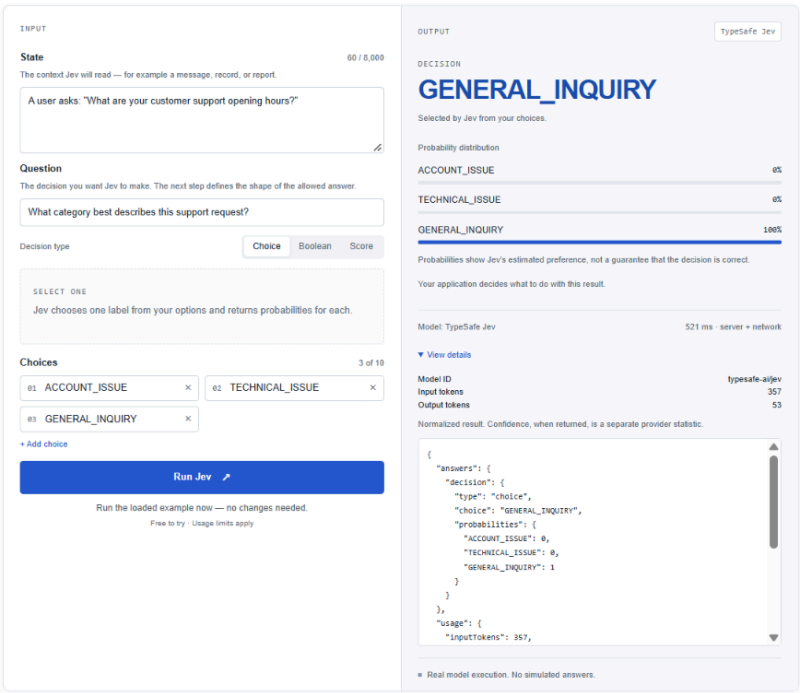


Figure 3. Jev classifies a question about support opening hours as GENERAL_INQUIRY.

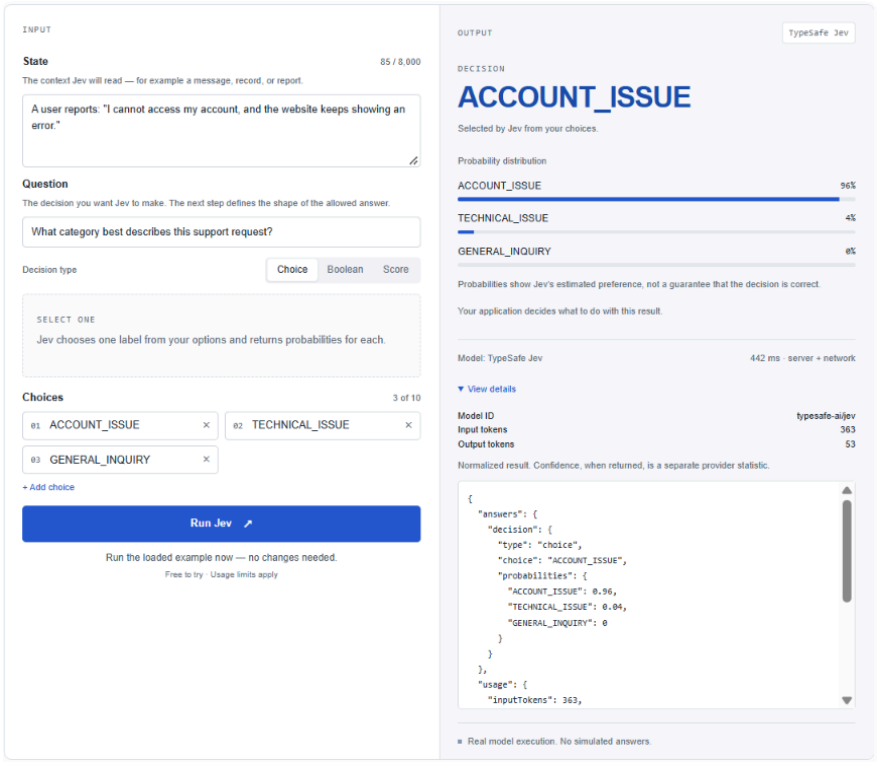


Figure 4. Jev selects ACCOUNT_ISSUE for a message containing both account and website

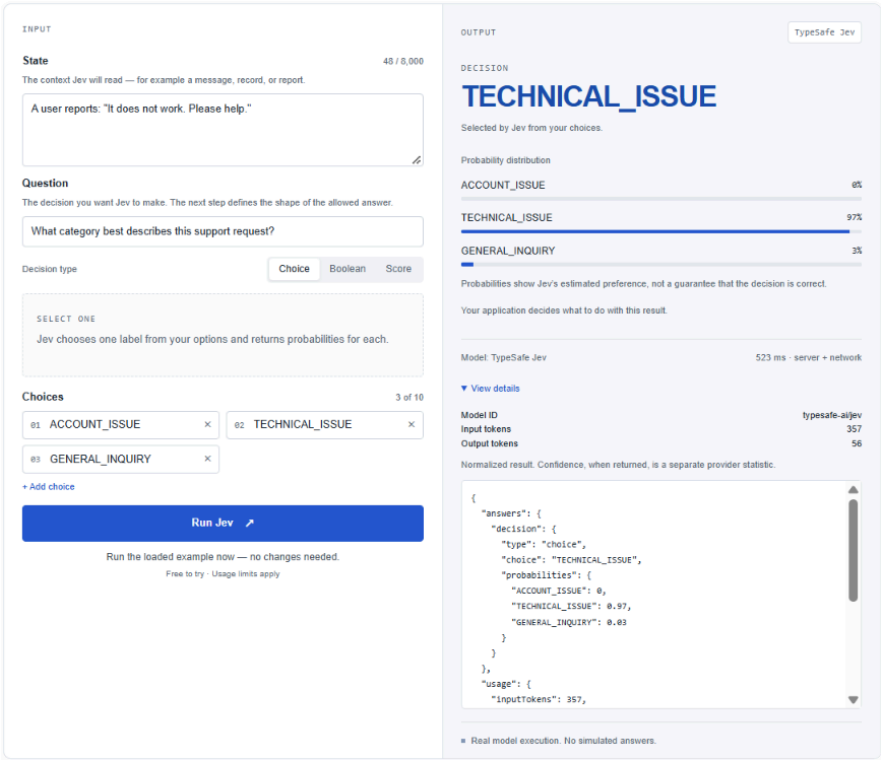


Figure 5. Jev selects TECHNICAL_ISSUE for a vague request that does not provide enough detail.

Output Schema

The model's decision must be one of the following predefined values:

- `ACCOUNT_ISSUE`
- `TECHNICAL_ISSUE`
- `GENERAL_INQUIRY`

The Python fallback validates the input and model result and determines whether the request can proceed or requires further handling.

3. Platform Integration

Proposed Architecture

The proposed integration consists of the following components:

1. **User:** Submits a support request.
2. **Support Application:** Receives the request and checks the input.
3. **Jev Decision Capability:** Classifies the support message into one of the three categories.
4. **Python Fallback Validation:** Checks the model result and identifies conditions requiring additional information or human review.
5. **Downstream Action:** Determines the next handling step based on the validation result.

The proposed workflow is:

User → Support Application → Jev Decision Capability → Python Fallback Validation → Downstream Action

The possible downstream actions are:

- `ROUTE_TO_SUPPORT` — Route a validated request to the appropriate support process.
- `ASK_FOR_MORE_INFORMATION` — Request additional details from the user.
- `SEND_TO_HUMAN` — Send the case for human review.

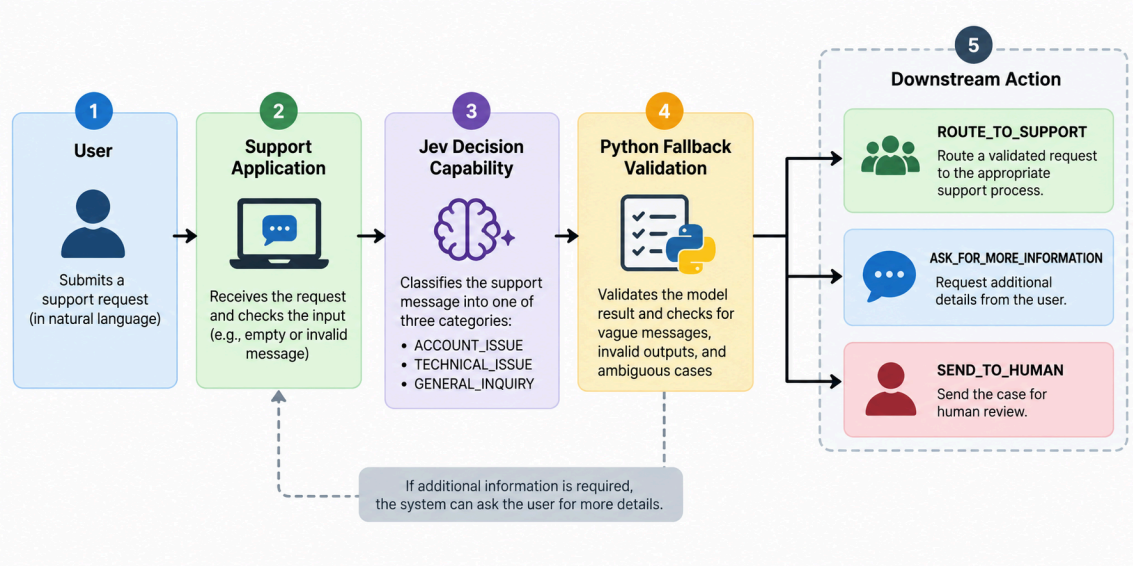


Figure 6. Proposed architecture of the Support Request Classifier and its fallback validation layer.

Implementation note: Jev Playground was used for the actual model experiments, while Python was used to implement and test the fallback logic. The diagram represents a proposed integration design; it does not indicate that a live API connection between Jev and Python has been deployed.

Example API Request

The following JSON illustrates a possible request format for a future application integration. It is a conceptual example, not evidence of a deployed API endpoint.

```
None
{
  "message": "The website shows a 500 Internal Server Error."
}
```

Example Model Response

An example of the structured decision returned by the model is:

```
None
{
```

```
  "decision": "TECHNICAL_ISSUE"
}
```

Example Validated Application Response

If the model result passes the application's validation rules, the proposed application response could be:

```
None
{
  "status": "ACCEPTED",
  "category": "TECHNICAL_ISSUE",
  "action": "ROUTE_TO_SUPPORT"
}
```

These examples illustrate a possible interface between the model and an application. They are illustrative JSON examples and do not demonstrate a working API integration.

Validation Rules

The Python fallback implements deterministic rules to handle invalid or ambiguous requests.

1. **Model error or timeout:** Return `REVIEW` with the action `SEND_TO_HUMAN`.
2. **Empty or invalid user message:** Return `REVIEW` with the action `ASK_FOR_MORE_INFORMATION`.
3. **Invalid model response format or unknown category:** Return `REVIEW` with the action `SEND_TO_HUMAN`.
4. **Vague request:** Return `REVIEW` with the action `ASK_FOR_MORE_INFORMATION`.
5. **Multiple issue indicators:** If both account-related and technical problem indicators are detected, return `REVIEW` with the action `SEND_TO_HUMAN`.
6. **Password recovery page failure:** If the relevant password reset or recovery page is reported as failing to load, return `REVIEW` with the action `SEND_TO_HUMAN`.
7. **Other valid requests:** Return `ACCEPTED` with the validated category and the action `ROUTE_TO_SUPPORT`.

These rules are implemented using deterministic checks in `fallback.py`. They are intended to reduce the risk of automatically routing requests when the model result is invalid or the message needs further review.

Fallback Demonstration

The fallback implementation is contained in `fallback.py`, and its tests are contained in `test_fallback.py`.

The recorded tests cover four scenarios:

- A vague support request.
- An unavailable or missing AI result.
- An unknown category.
- A message containing multiple issue types.

The recorded test output should be included as evidence of the fallback behavior.

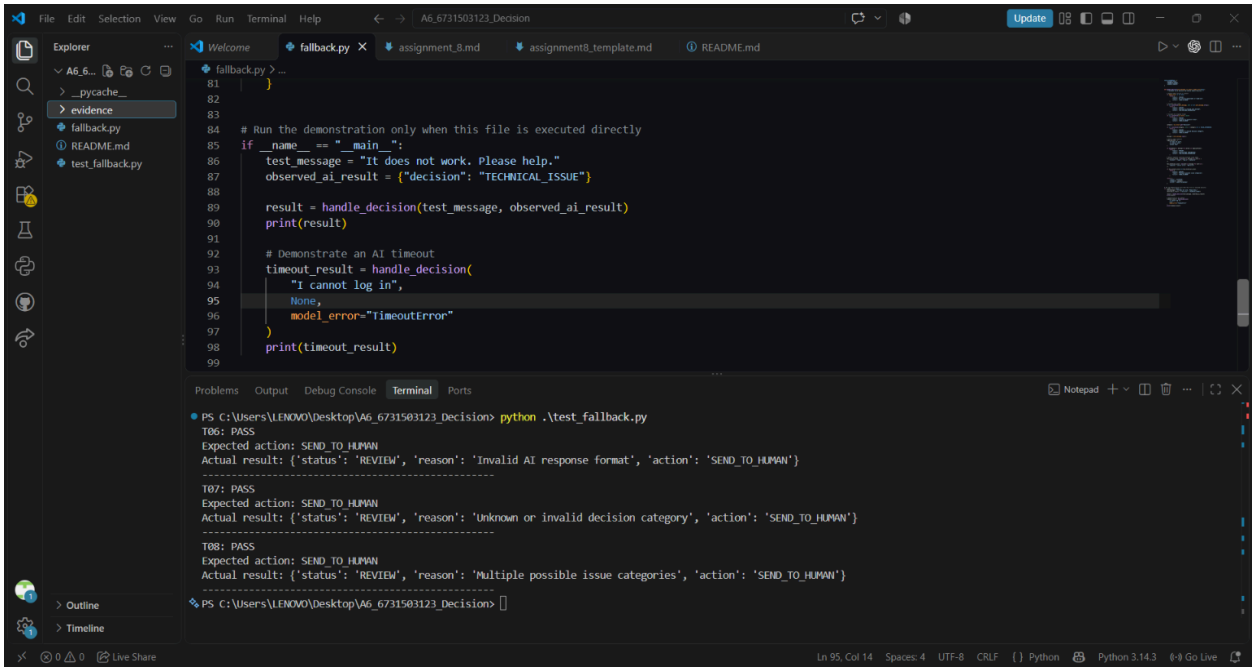


Figure 7. Python fallback tests in the development environment.

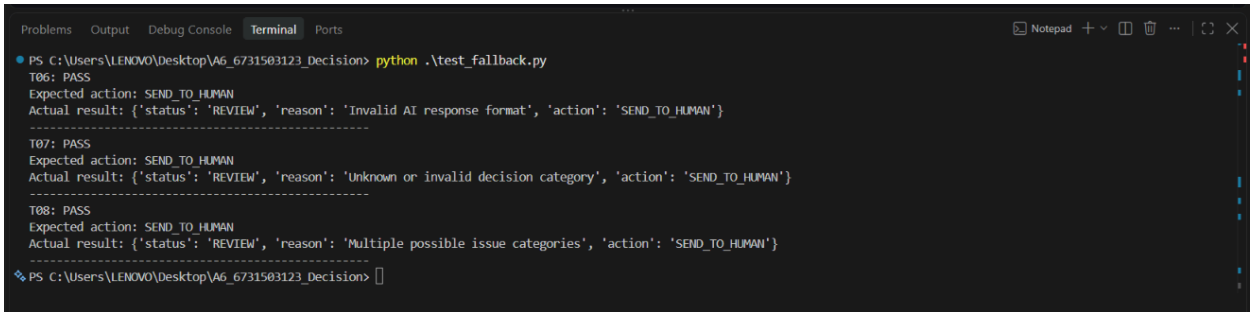


Figure 8. Recorded results of the Python fallback tests.

The fallback tests demonstrate the behavior of the local Python validation logic. They do not measure the classification accuracy of the Jev model itself.

4. Engineering Reflection

What Worked Well

Jev successfully categorized three clear support requests into account issues, technical issues, and general inquiries. The Python fallback added a deterministic validation layer for handling invalid results and ambiguous requests.

Using both an AI decision capability and explicit fallback rules provides a clearer handling process than relying on the model's category alone.

Unexpected Behavior or Mistakes

For T04, Jev selected `ACCOUNT_ISSUE` even though the message also described a website error. For T05, Jev selected `TECHNICAL_ISSUE` even though the request did not provide enough information to identify the actual problem.

These results show that a model's predicted category and confidence do not guarantee that the message contains sufficient information for safe routing.

Production Risks

Incorrect classification could send support requests to the wrong team and delay resolution. Other risks include vague messages, overlapping categories, unavailable AI services, invalid outputs, and over-reliance on model confidence.

The current fallback uses simple English phrase matching. Therefore, it may not recognize every paraphrase or messages written in other languages.

Before production use, the system would require broader testing, monitoring, improved handling of varied wording, and human review for uncertain cases.

Next Improvement

The next step is to extend the capability with a small grounding knowledge base and a retrieval mechanism, evaluate a fixed set of ten cases, and improve one specific part of the handling of ambiguous requests.

The evaluation should distinguish actual AI model outputs from simulated results used to test the local fallback pipeline. This distinction is necessary to report the system's performance accurately.

5. References and AI Assistance

- **Assignment A6:** *Build and Integrate an AI Decision Capability*, provided by the instructor.
- **A6 Individual Submission Template:** Provided by the instructor.
- **Jev Playground:** Used to run the five support request classification experiments and collect actual model outputs. Available at: https://jevplayground.com/
- **Jev Documentation:** Documentation used to understand the Playground and decision workflow. Available at: https://jevplayground.com/how-to-use-jev
- **Project source files:** `fallback.py` and `test_fallback.py`.
- **AI Assistance Disclosure:** The project implementation, testing, and evidence collection were carried out by the student. ChatGPT was used as a supplementary tool for minor assistance with improving report wording, clarifying technical concepts, and reviewing the explanation of the validation and fallback logic.

6. Submission Checklist

Before submitting, verify the following:

- ☑ Decision configuration or relevant source code is included as required.
- ☑ Five unique test inputs and their actual model outputs are documented.
- ☑ The architecture diagram is complete and matches the proposed design.
- ☑ Example request and response formats are included and clearly identified as conceptual.
- ☑ Validation and fallback rules are explained accurately.
- ☑ Fallback test evidence is included and readable.
- ☑ Engineering reflection is complete.
- ☑ References and AI assistance disclosure are included.
- ☑ Student name and student ID are correct.

- [x] All figure placeholders have been replaced with the appropriate images or evidence.
- [x] The final PDF is saved as `A6_6731503123.pdf`.